Design/Implementation Phase
Progress Report

on

AGENT RELIABILITY LAB

Controlled Fault Injection and Evidence-Based
Failure Analysis for AI Agents

Capstone Project 1 (BCSE0758), Semester VII
Bachelor of Technology in Computer Science and Engineering

Submitted by

Name	Roll number	Role
Suryansh Singh	2301330100208	Team lead
Simran Kumari	2301330100200	Member
Tanishka Sahu	2301330100210	Member

Group CSE/CSPROJECT/2027/G053

Under the supervision of
Mr. Pradeep Kumar (Assistant Professor, CSE)

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY
GREATER NOIDA
(An Autonomous Institute)
Department of Computer Science and Engineering

Session 2026-27

Progress review: 17 October 2026

Contents

1 Proposed System **1**
- 1.1 Background and Problem . . . 1
- 1.2 Objectives and Status . . . 1
- 1.3 Overview of the Proposed System . . . 1

2 Software Requirement Analysis **3**
- 2.1 Functional Requirements . . . 3
- 2.2 Non-Functional Requirements . . . 4
- 2.3 Major Modules and Their Functionalities . . . 4

3 Software Design **6**
- 3.1 System Architecture . . . 6
- 3.2 Class Design . . . 6
- 3.3 Sequence Diagram . . . 8
- 3.4 Data Flow Diagrams . . . 8
- 3.5 Database Design . . . 9
- 3.6 Project Schedule . . . 10

4 Implementation of Core Modules **11**
- 4.1 Algorithms . . . 11
  - 4.1.1 Seeded, Agent-Independent Fault Injection . . . 11
  - 4.1.2 Repeated Trials and Reliability Estimators . . . 11
  - 4.1.3 Failure Classification . . . 11
  - 4.1.4 Paired Comparison and Configuration Search . . . 12
- 4.2 Code Snapshots . . . 12
- 4.3 Testing and Quality . . . 15
- 4.4 Implementation Output . . . 16

5 Preliminary Results **17**

6 Research Paper, Work Distribution, and Next Steps **19**
- 6.1 First Research Paper . . . 19
- 6.2 Work Distribution . . . 19
- 6.3 Next Steps . . . 20

A Annexure-2 Details (Project Progress Evaluation Report) **22**

B Work Log from Version History **23**

List of Figures

3.1 System architecture. Only the execution layer calls agents. 6
3.2 Class diagram 7
3.3 Sequence diagram of `relai run` 8
3.4 Level 0 data flow diagram (context) 8
3.5 Level 1 data flow diagram 9
3.6 Entity-relationship diagram of the run store 9
3.7 Gantt chart. Green bars are complete; amber bars are planned. 10

4.1 Failure classification flowchart 12
4.2 Fault decision for one tool call (`env.py`) 12
4.3 Evaluation loop with trials and crash capture (`harness.py`) 13
4.4 Recovery episodes (`analysis.py`) 13
4.5 Retry policy of the reference agent (`baselines.py`) 14
4.6 Pareto front (`optimize.py`) 14
4.7 A three-call task in the reliability suite 14
4.8 Test and lint output 15
4.9 Three agents under 30% tool timeouts, five trials per task 16

5.1 Success and pass^5 as the timeout rate rises 17
5.2 Measured success against $(1-p^{r+1})^L$ for L tool calls and r retries, 200 trials per task 18

List of Tables

1.1 Project objectives and their status at the progress review 1

2.1 Functional requirements 3
2.2 Non-functional requirements and how they are met 4
2.3 Modules, responsibilities, and owners 4

4.1 Automated tests 15

5.1 Agents under 30% tool timeouts 17
5.2 Classifier self-check: diagnosed root cause per planted defect (60 runs each) 18

6.1 Status of the research paper sections 19
6.2 Ownership by member 19

CHAPTER 1

Proposed System

1.1 Background and Problem

Agents built on large language models (LLMs) plan, call tools, read observations, and retry after errors. Benchmarks usually report one number for them: did the agent finish the task? That number hides three things a team needs before it deploys an agent. It hides *why* a run failed, for example a skipped tool, a forbidden tool, a repeated call, or a timeout that was never retried. It hides *consistency*: an agent that succeeds 77% of the time may solve only a third of its tasks in all of five attempts. And it hides *cost*: a more reliable setting may need twice the spend and latency.

The synopsis proposed Agent Reliability Lab as a small, inspectable Python laboratory for this problem. During the design and implementation phase the idea was sharpened into *a crash-test lab for AI agents*: tools are broken on purpose in a repeatable way, every agent is run several times, every failed run is explained with evidence, and the cheapest setup that stays reliable is selected.

1.2 Objectives and Status

Table 1.1: Project objectives and their status at the progress review

Objective	What was built	Status
O1 Evaluate agents	Success, correctness, cost, latency, tokens, tool calls, recovery, steps; pass@k, pass^k, and consistency over repeated trials	Done
O2 Explain failures and recovery	Seeded fault injection; five failure modes with evidence; unclassified outcome; recovery scoring; planted-defect self-check	Done
O3 Controlled comparison and optimization	Identical fault schedules for all variants; Wilson and bootstrap intervals; exact McNemar test; grid sweep, Pareto front, recommendation	Done; live LLM study next

1.3 Overview of the Proposed System

The system is a Python package, `relai`, with a command-line interface. A researcher writes or picks a task suite, wraps an agent behind a two-member interface (a name and

a `run(task)` method that returns a trajectory), and runs an experiment. The harness runs every variant on every task for k trials through a tool environment that injects faults on a seeded schedule. Scoring, metrics, and failure analysis read the recorded trajectories. Statistics, configuration search, storage, and reports operate on the results. Fig. 3.1 in Chapter 3 shows the architecture.

CHAPTER 2

Software Requirement Analysis

2.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	Load task suites from YAML or JSON files, including tasks, expected answers, permitted tools, tags, step budgets, mock tools, default faults, and reference tool plans.
FR2	Accept any agent that exposes a name and `run(task)` returning a trajectory, including plain functions, scripted traces, offline baseline agents, and LLMs behind an OpenAI-compatible API.
FR3	Inject reproducible tool faults (timeouts, outages, fail-first-N) whose schedule depends only on seed, task, trial, tool, and call number.
FR4	Run each task for k trials and report success, pass@k, pass^k, and consistency.
FR5	Score answers with exact, contains, normalized, numeric (with tolerance and partial credit), and regular-expression scorers.
FR6	Derive cost, wall and step latency, tokens, tool calls, repeated calls, recovery rate, and step count from each trajectory.
FR7	Classify each run into planning, tool-misuse, context, recovery, and horizon failure modes with evidence and a root cause; mark unmatched failures as unclassified.
FR8	Compare variants with confidence intervals and an exact paired McNemar test.
FR9	Sweep agent settings over a grid, return the Pareto front over success, cost, and latency, and recommend the cheapest setting that meets a success target.
FR10	Store experiments in SQLite and JSON Lines; reload them losslessly; render HTML and Markdown reports.
FR11	Check the failure classifier against agents with planted defects.
FR12	Expose every capability through commands (`demo`, `suites`, `run`, `sweep`, `stress`, `selfcheck`, `report`) and a Python API.

2.2 Non-Functional Requirements

Table 2.2: Non-functional requirements and how they are met

Quality	Requirement and evidence
Reproducibility	The same seed yields the same faults and numbers. Ranking breaks ties deterministically. Every figure in this report is regenerated by one script.
Fairness	Every variant meets the identical fault schedule, so runs are paired and comparisons are controlled.
Performance	About 6,500 fully scored and classified simulated runs per second on a laptop; the 81 tests run in under one second.
Portability	Python 3.11 to 3.13 on Linux, macOS, or Windows; two runtime dependencies (Pydantic, PyYAML); no GPU or network needed.
Extensibility	New agents, scorers, suites, fault kinds, and failure modes attach without changing the harness.
Robustness	An exception inside an agent becomes a failed run with an error step instead of stopping the experiment.
Security	No `eval`: the calculator tool parses arithmetic with an AST whitelist; HTML reports escape all trajectory text; offline by default.
Maintainability	Typed Pydantic models, one test file per module, Ruff lint, continuous integration on three Python versions.
Usability	One command per task, tables with intervals in the terminal, standalone HTML reports.

2.3 Major Modules and Their Functionalities

Table 2.3: Modules, responsibilities, and owners

Module (file)	Functionality	Owner
Domain types (`types.py`)	Task, Step, Trajectory, Score, RunMetrics, FailureReport, RunResult	Suryansh
Agent seam (`agents.py`)	Agent protocol, FunctionAgent, ScriptedAgent	Suryansh

Module (file)	Functionality	Owner
Tool environment (`env.py`)	Tools, seeded FaultPolicy, per-run ToolSession, Recorder, lookup and calculator tools	Suryansh
Harness (`harness.py`)	Trials, timing, crash capture, pass@k, pass^k, consistency	Suryansh
Experiments (`experiment.py`)	Variants, controlled runs, stress tests, ranking	Suryansh
Metrics (`metrics.py`)	Scorers and per-run metric derivation	Simran
Failure analysis (`analysis.py`)	Five detectors, error episodes, root cause, evidence	Simran
Statistics (`stats.py`)	Wilson, bootstrap, task-clustered bootstrap, exact McNemar	Simran
Optimizer (`optimize.py`)	Grid sweep, dominance, Pareto front, recommendation	Simran
Suites (`suite.py`, `suites/`)	YAML/JSON loading, validation, built-in suites	Tanishka
Baselines (`baselines.py`)	Naive, retry, guessing, looping, careless agents	Tanishka
Self-check (`selfcheck.py`)	Classifier check against planted defects	Tanishka
Reports (`report.py`)	HTML and Markdown rendering	Tanishka
Store (`store.py`)	SQLite and JSON Lines persistence	Suryansh
CLI (`cli.py`)	The `relai` command	Suryansh
LLM adapter (`adapters/`)	OpenAI-compatible tool-calling agent	Suryansh
Tests and examples	81 tests in 13 files; five runnable examples	Tanishka

CHAPTER 3

Software Design

3.1 System Architecture

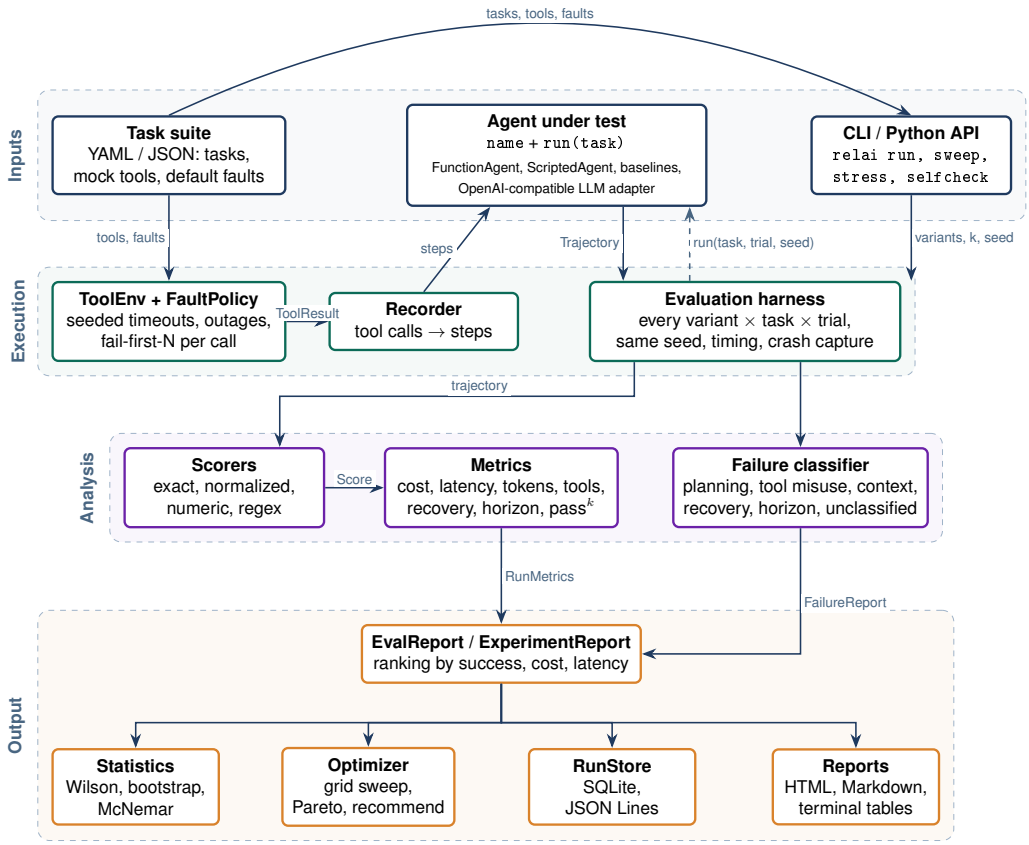


Figure 3.1: System architecture. Only the execution layer calls agents.

3.2 Class Design

Fig. 3.2 shows the classes. The data model on the left describes one run. RunResult composes the trajectory with its Score, RunMetrics, and FailureReport; EvalReport groups the runs of one variant and ExperimentReport groups variants. Every agent implements the Agent protocol; the baseline agents share PolicyAgent, and LoopingAgent and CarelessAgent extend RetryAgent so that each carries exactly one defect.

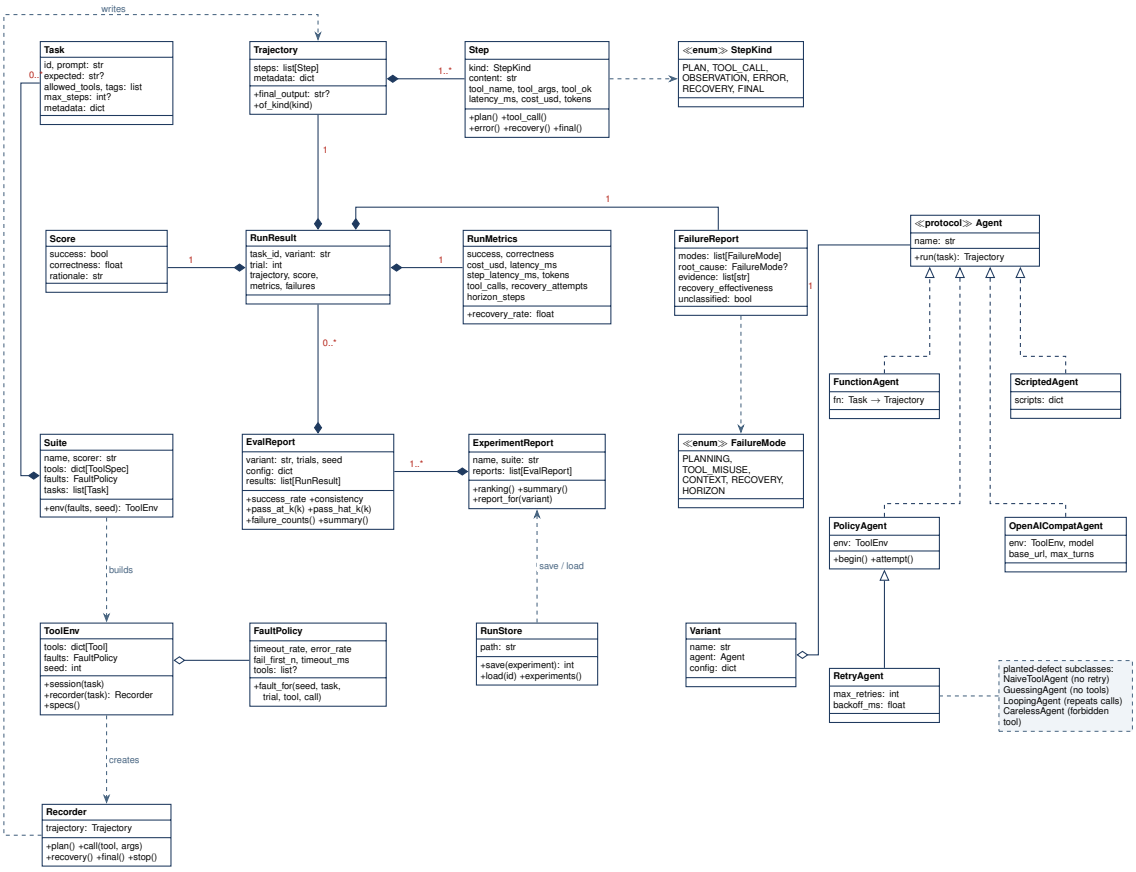


Figure 3.2: Class diagram

3.3 Sequence Diagram

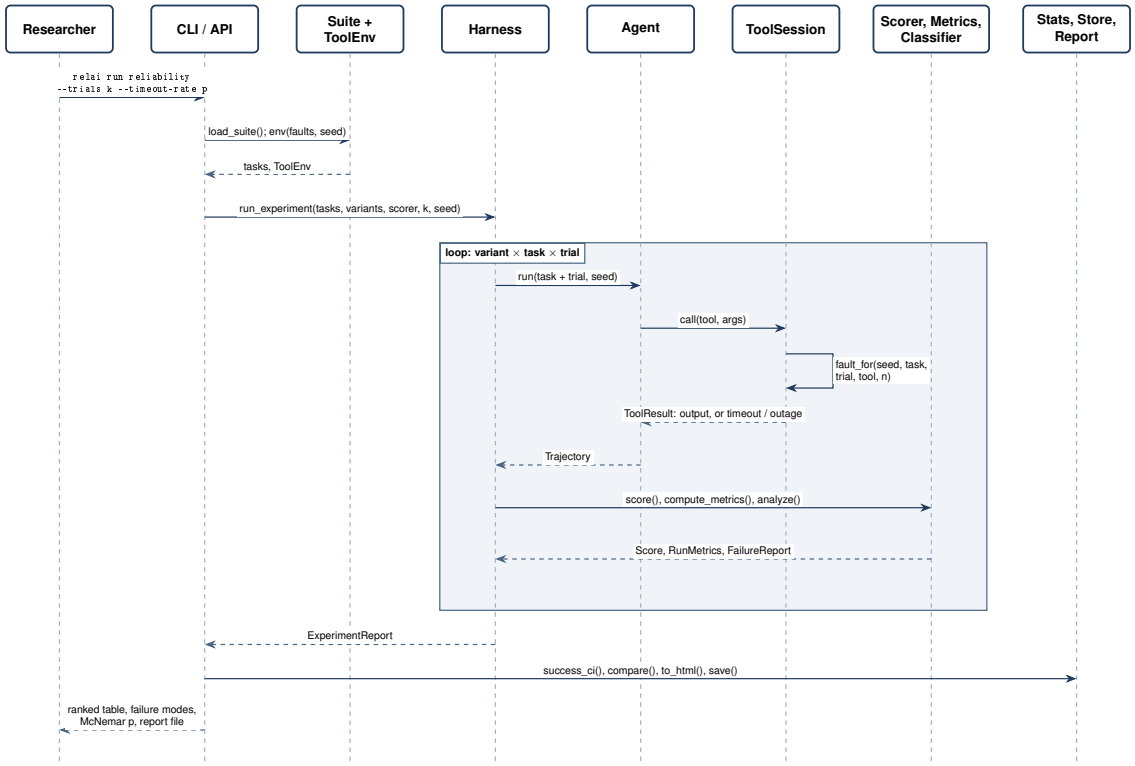


Figure 3.3: Sequence diagram of `relai run`

3.4 Data Flow Diagrams

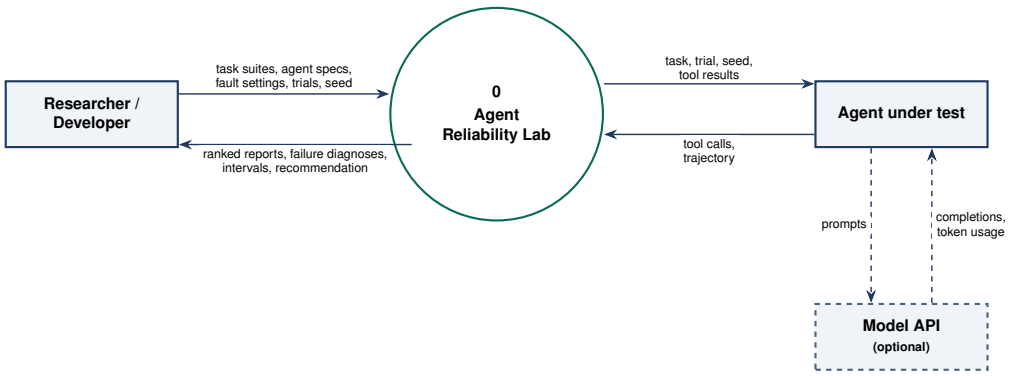


Figure 3.4: Level 0 data flow diagram (context)

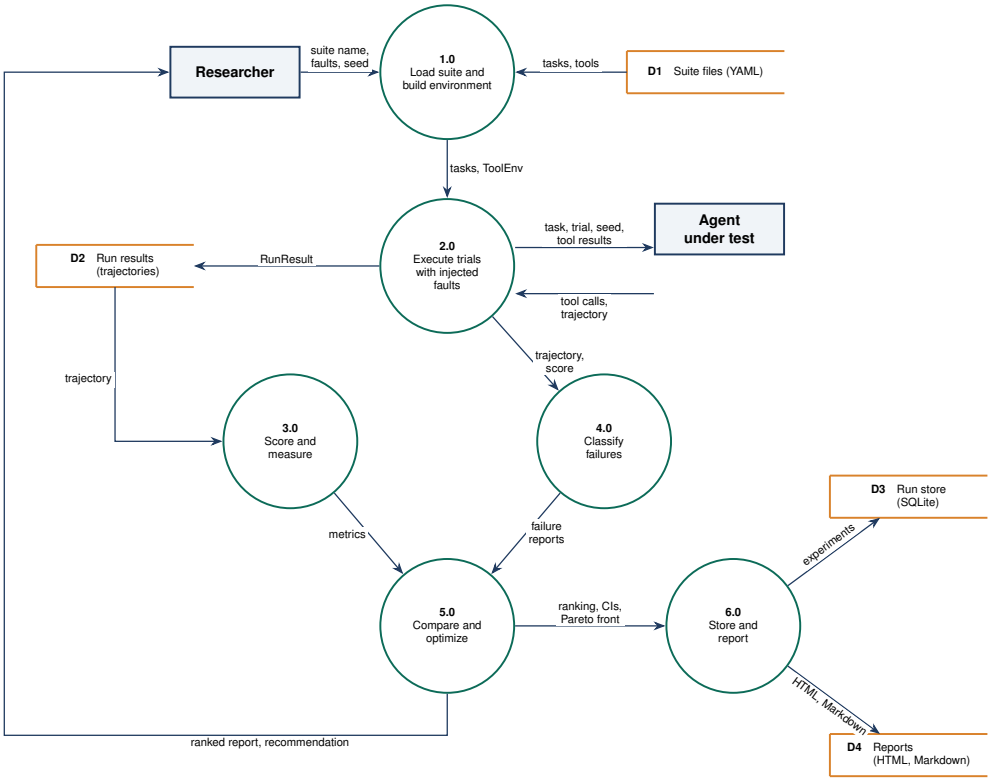


Figure 3.5: Level 1 data flow diagram

3.5 Database Design

Results persist in a SQLite database whose entities mirror the in-memory reports. Each run row keeps the complete run as JSON for lossless reloading; the steps and failures tables exist for SQL queries such as failure counts per variant.

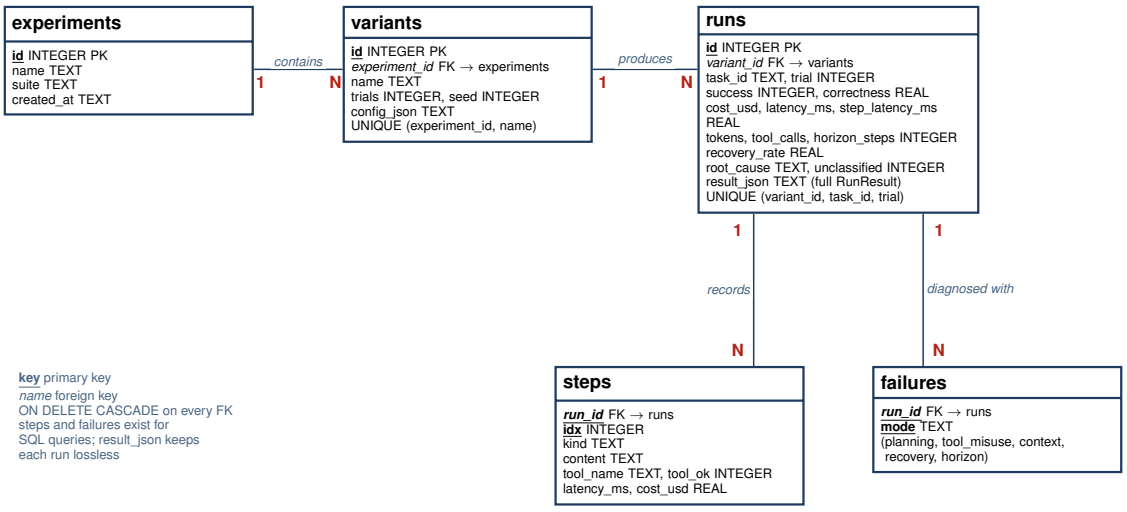


Figure 3.6: Entity-relationship diagram of the run store

3.6 Project Schedule

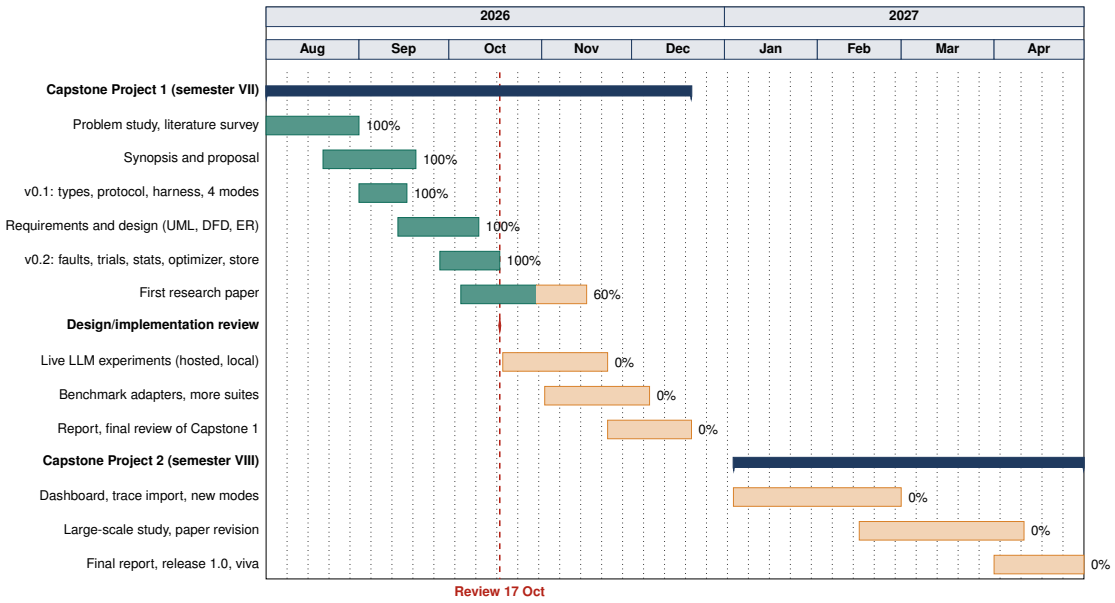


Figure 3.7: Gantt chart. Green bars are complete; amber bars are planned.

CHAPTER 4

Implementation of Core Modules

4.1 Algorithms

4.1.1 Seeded, Agent-Independent Fault Injection

For the n-th call to tool τ in trial i of task t under seed s, the environment computes $u = h(s,t,i,\tau,n) \in [0,1)$ with a seeded pseudo-random generator. The call times out if n is within the fail-first window or $u < p_{\text{to}}$, returns an outage if $p_{\text{to}} \le u < p_{\text{to}} + p_{\text{err}}$, and otherwise runs the tool. Because h ignores the agent, every variant meets the same failures, which makes paired tests valid. Unknown tools and invalid arguments fail without an error step and are therefore recorded as misuse by the agent.

4.1.2 Repeated Trials and Reliability Estimators

With c successes in n trials of a task, $\text{pass@}k = 1 - \binom{n-c}{k} / \binom{n}{k}$ and $\text{pass}^k = \binom{c}{k} / \binom{n}{k}$, averaged over tasks. A task is consistent when all of its trials agree.

4.1.3 Failure Classification

1. Run five detectors (planning, tool misuse, context, recovery, horizon); each returns the index of its first evidence and a reason.
2. An error is recovered when a later call of the failed tool succeeds or the run ends with a correct answer; failed retries in between do not close the episode.
3. If any mode matched, the root cause is the one with the earliest evidence.
4. If none matched and the run failed, mark it unclassified; never force a label.

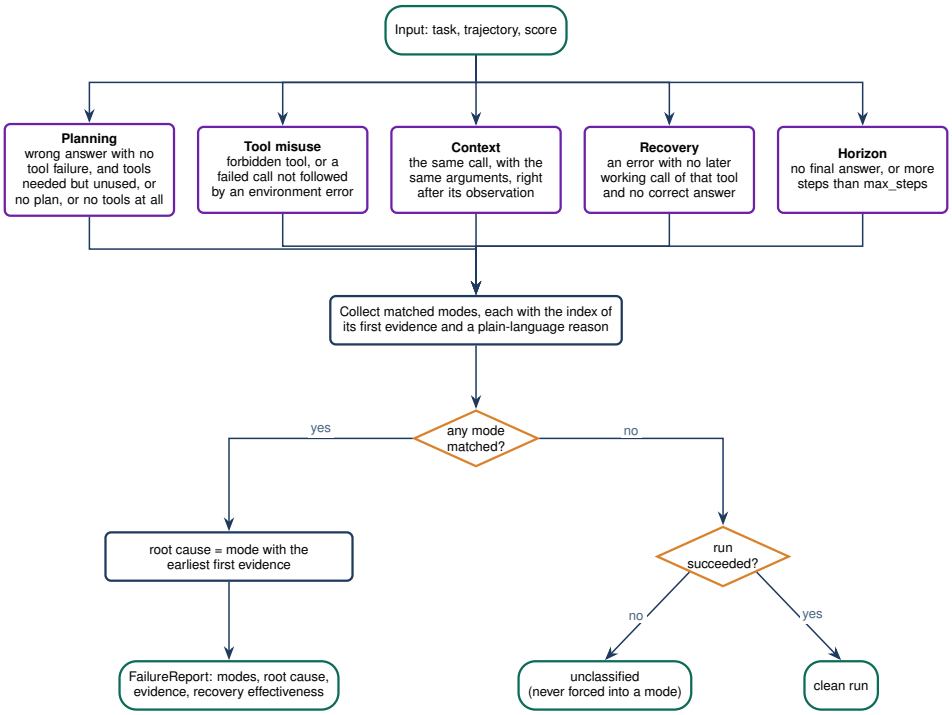


Figure 4.1: Failure classification flowchart

4.1.4 Paired Comparison and Configuration Search

Runs of two variants are paired by (task, trial). With b runs passed only by the first variant and c only by the second, the exact McNemar p-value is $\min\left(1, 2\sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 2^{-(b+c)}\right)$. A sweep evaluates one variant per grid point; the Pareto front keeps variants not dominated on success, cost, and latency, and the recommendation is the cheapest variant whose success meets the target.

4.2 Code Snapshots

src/relai/env.py (line 46)

```python
def fault_for(self, *, seed: int, task_id: str, trial: int, tool: str, call: int) -> str | None:
    """Return the fault for the call-th call of a tool in one run, if any."""
    if not self.applies_to(tool):
        return None
    if call <= self.fail_first_n:
        return "timeout"
    draw = random.Random(f"{seed}:{task_id}:{trial}:{tool}:{call}").random()
    if draw < self.timeout_rate:
        return "timeout"
    if draw < self.timeout_rate + self.error_rate:
        return "unavailable"
    return None
```

Figure 4.2: Fault decision for one tool call (`env.py`)

src/relai/harness.py (line 141)

```python
def evaluate(
    agent: Agent,
    tasks: Sequence[Task],
    scorer: Scorer = exact_match,
    *,
    trials: int = 1,
    seed: int = 0,
) -> EvalReport:
    """Run one agent on every task, trials times each, and return scored reports.

    The trial index and seed reach the agent through task.metadata, so tool
    environments can replay the same fault schedule for every variant.
    """
    if trials < 1:
        raise ValueError("trials must be at least 1")
    results: list[RunResult] = []
    for task in tasks:
        for trial in range(trials):
            run_task = task.model_copy(
                update={"metadata": {**task.metadata, TRIAL_KEY: trial, SEED_KEY: seed}},
                deep=True,
            )
            started = time.perf_counter()
            trajectory = _run_safely(agent, run_task)
            wall_ms = (time.perf_counter() - started) * 1000
            score = scorer(task, trajectory)
            metrics = compute_metrics(trajectory, score, wall_ms)
            failures = analyze(task, trajectory, score)
            results.append(
                RunResult(
                    task_id=task.id,
                    variant=agent.name,
                    trial=trial,
                    trajectory=trajectory,
                    score=score,
                    metrics=metrics,
                    failures=failures,
                )
            )
    return EvalReport(variant=agent.name, trials=trials, seed=seed, results=results)
```

Figure 4.3: Evaluation loop with trials and crash capture (`harness.py`)

src/relai/analysis.py (line 160)

```python
def error_episodes(trajectory: Trajectory, score: Score) -> list[tuple[int, bool]]:
    """Each error step's index and whether the run later restored a working path.

    A path is restored by a later successful call of the tool that failed, or of
    any tool when the error follows no tool call, or by a correct final answer.
    Failed retries in between do not break the episode, so a chain of retries
    that ends in success counts every one of its errors as recovered.
    """
    steps = trajectory.steps
    episodes: list[tuple[int, bool]] = []
    for idx, step in enumerate(steps):
        if step.kind != StepKind.ERROR:
            continue
        previous = steps[idx - 1] if idx else None
        failed_tool = (
            previous.tool_name
            if previous is not None and previous.kind == StepKind.TOOL_CALL
            else None
        )
        recovered = False
        for later in steps[idx + 1 :]:
            if (
                later.kind == StepKind.TOOL_CALL
                and later.tool_ok
                and (failed_tool is None or later.tool_name == failed_tool)
            ):
                recovered = True
                break
            if later.kind == StepKind.FINAL:
                recovered = score.success
                break
        episodes.append((idx, recovered))
    return episodes
```

Figure 4.4: Recovery episodes (`analysis.py`)

src/relai/baselines.py (line 91)

```
def attempt(self, rec: Recorder, tool: str, args: dict[str, Any]) -> ToolResult:
    result = rec.call(tool, args)
    retries = 0
    while not result.ok and result.fault is not None and retries < self.max_retries:
        retries += 1
        rec.recovery(
            f"retry {retries}/{self.max_retries} after {result.fault}",
            latency_ms=self.backoff_ms * 2 ** (retries - 1),
        )
        result = rec.call(tool, args)
    return result
```

Figure 4.5: Retry policy of the reference agent (`baselines.py`)

src/relai/optimize.py (line 81)

```
def pareto_front(
    reports: Sequence[EvalReport],
    objectives: Sequence[Objective] = DEFAULT_OBJECTIVES,
) -> list[EvalReport]:
    """Variants that no other variant beats on every objective at once."""
    front = [
        report
        for report in reports
        if not any(dominates(other, report, objectives) for other in reports if other is not report)
    ]
    return ExperimentReport(reports=front).ranking()
```

Figure 4.6: Pareto front (`optimize.py`)

src/relai/suites/reliability.yaml

```
      - {tool: calculator, args: {expression: "{0} + {1}"}}
  - id: latest_order_status
    prompt: What is the status of the latest order placed by ravi@example.com?
    expected: delivered
    allowed_tools: [customer_lookup, order_history, order_status]
    tags: [requires_tools, long-horizon]
    max_steps: 12
    guess: shipped
    solution:
      - {tool: customer_lookup, args: {email: ravi@example.com}}
      - {tool: order_history, args: {customer: "{0}"}}
      - {tool: order_status, args: {order: "{1}"}}
```

Figure 4.7: A three-call task in the reliability suite

4.3 Testing and Quality

Table 4.1: Automated tests

Test file	Tests	What it pins
`test_analysis.py`	13	Each failure mode, when it fires and when it must stay silent; retry chains; unclassified
`test_env.py`	10	Fault reproducibility across agents, trials, and seeds; misuse vs. environment errors; safe calculator
`test_harness.py`	5	Trials, pass@k, pass^k, consistency, crash capture
`test_metrics.py`	9	Scorers and metric derivation
`test_stats.py`	5	Wilson values, seeded bootstrap, McNemar p-values, pairing
`test_optimize.py`	5	Grid, dominance, Pareto front, recommendation, cost-reliability trade-off
`test_baselines.py`	8	Each planted defect is diagnosed; self-check agreement
`test_suite.py`, `test_store.py`, `test_report.py`	12	Suite validation and round trips; SQLite and JSONL round trips; HTML escaping
`test_adapter.py`, `test_cli.py`, `test_experiment.py`	14	LLM tool loop with a fake transport; every CLI command; ranking
Total	**81**	All pass; Ruff lint clean; CI on Python 3.11, 3.12, 3.13

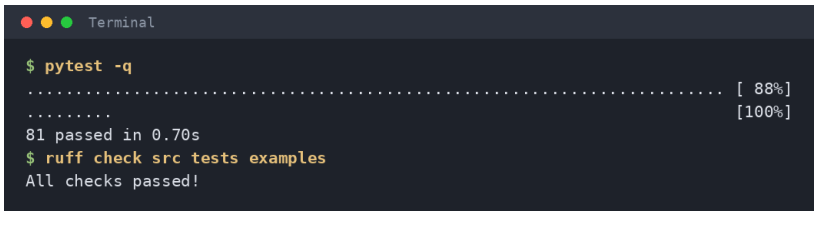


Figure 4.8: Test and lint output

4.4 Implementation Output

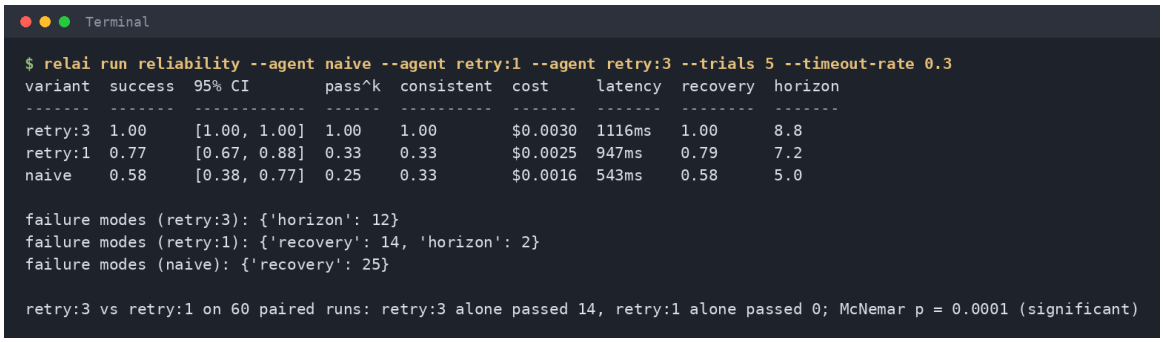


Figure 4.9: Three agents under 30% tool timeouts, five trials per task

CHAPTER 5

Preliminary Results

All results use offline reference agents, so the ground truth of each run is known. They validate the instrument; they do not rank LLMs. Unless noted, the suite has twelve tasks, each run five times with seed 0 and a tool timeout rate of 0.3.

Table 5.1: Agents under 30% tool timeouts

Agent	Success	95% CI	pass^5	Cost (USD)	Lat. (ms)	Root causes
naive	0.58	[0.38, 0.77]	0.25	0.0016	543	25 recovery
retry, 1	0.77	[0.67, 0.88]	0.33	0.0025	947	14 recov., 2 horiz.
retry, 3	1.00	[1.00, 1.00]	1.00	0.0030	1116	12 horizon

Three retries beat one retry on 14 paired runs and lose on none (exact McNemar $p = 1.2 \times 10^{-4}$).

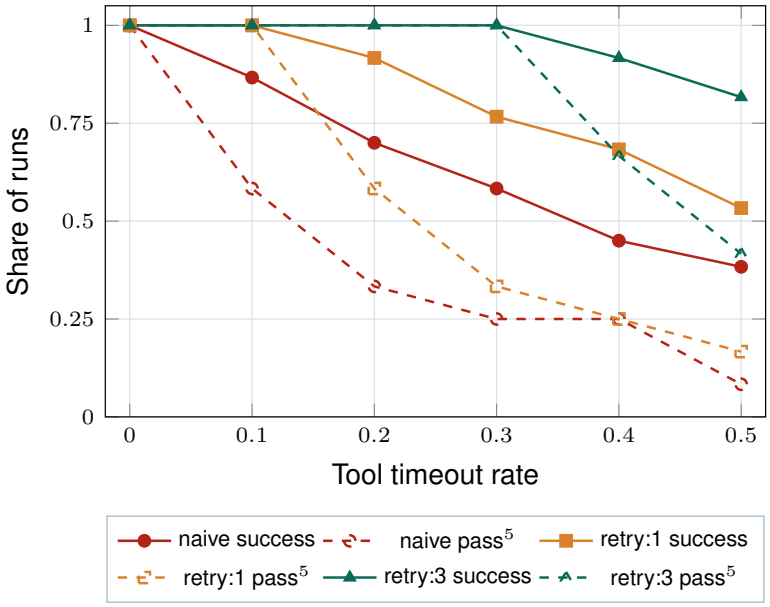


Figure 5.1: Success and pass^5 as the timeout rate rises

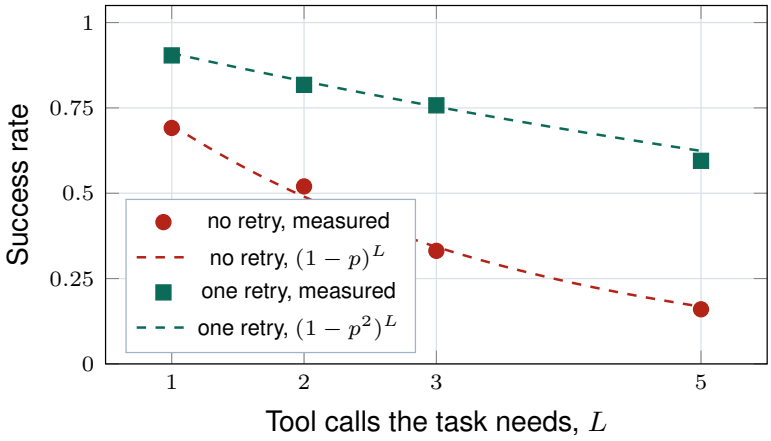


Figure 5.2: Measured success against $(1-p^{r+1})^L$ for L tool calls and r retries, 200 trials per task

Table 5.2: Classifier self-check: diagnosed root cause per planted defect (60 runs each)

Agent	Planted	Planning	Misuse	Context	Recovery	Horizon	None
naive	recovery	0	0	0	25	0	35
guessing	planning	55	0	0	0	0	5
looping	context	0	0	55	0	0	5
careless	tool misuse	0	60	0	0	0	0
retry	control	0	0	0	1	11	48

Precision is 1.00 for all four planted modes. A sweep over zero to four retries places every setting on the Pareto front; two retries is the cheapest setting with success of at least 0.95 (success 0.98, pass^5 0.92).

CHAPTER 6

Research Paper, Work Distribution, and Next Steps

6.1 First Research Paper

Title: Agent Reliability Lab: Controlled Fault Injection and Evidence-Based Failure Analysis for Tool-Using Language Agents. Prepared in the prescribed IEEE conference template (A4).

Table 6.1: Status of the research paper sections

Section	Status
Abstract	Complete
Introduction	Complete, with five contributions
Related work	Complete: benchmarks, reliability metrics, fault injection, failure attribution, frameworks
Problem statement	Complete, formal
Proposed methodology	Complete: fault model, estimators, taxonomy, statistics, search
Preliminary results	Complete for reference agents; live LLM results pending
References	25 sources, IEEE style

6.2 Work Distribution

Table 6.2: Ownership by member

Member	Owned work
Suryansh Singh (lead)	Architecture and design rules, domain types, agent seam, tool environment and fault injection, harness and trials, experiments, store, CLI, LLM adapter, integration, paper drafting
Simran Kumari	Metrics and scorers, failure taxonomy and analysis, statistics, optimizer, results analysis, related-work survey
Tanishka Sahu	Task suites, baseline agents, self-check, reports, tests and examples, diagrams, documentation

6.3 Next Steps

1. Run hosted and local LLMs through the adapter on the reliability suite and add the results to the paper.
2. Add suites adapted from public benchmarks and more fault kinds (rate limits, corrupted or truncated outputs).
3. Complete the Capstone Project 1 report and final review (December 2026).
4. Semester VIII: dashboard, import of traces from tracing platforms, larger study, release 1.0.

Bibliography

[1] S. Yao, J. Zhao, D. Yu et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[2] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," arXiv:2406.12045, 2024.

[3] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," arXiv:2407.01502, 2024.

[4] S. Kapoor et al., "Holistic agent leaderboard: The missing infrastructure for AI agent evaluation," in *Proc. ICLR*, 2026.

[5] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, and A. Narayanan, "Towards a science of AI agent reliability," in *Proc. ICML*, PMLR vol. 306, 2026.

[6] M. Cemri et al., "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.

[7] S. Zhang et al., "Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems," in *Proc. ICML*, PMLR vol. 267, 2025.

[8] "ToolMisuseBench: An offline deterministic benchmark for tool misuse and recovery in agentic systems," arXiv:2604.01508, 2026.

[9] "AgentChaos: Chaos engineering for agent systems via programmatic fault injection," arXiv:2608.06790, 2026.

[10] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[12] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[13] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[14] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.

CHAPTER A

Annexure-2 Details (Project Progress Evaluation Report)

The fields below are pre-filled for the printed Annexure-2 form. Grades, remarks, and signatures are left for the supervisor, coordinator, and committee.

School	School of Computer Science and Information Technology
Department	Computer Science and Engineering
Review	Review-1 (Design/Implementation Phase), 17 October 2026
Project type	Major Project (Capstone Project 1, BCSE0758)
Group number	G053 (CSE/CSPROJECT/2027/G053)
Session, year, semester	2026-27, fourth year, semester VII
Project title	Agent Reliability Lab: Controlled Fault Injection and Evidence-Based Failure Analysis for AI Agents
Supervisor	Mr. Pradeep Kumar, Assistant Professor, CSE
Co-supervisor	None

Sr. No.	Roll number	Name	Section	Signature
1	2301330100208	Suryansh Singh	C	_____
2	2301330100200	Simran Kumari	C	_____
3	2301330100210	Tanishka Sahu	C	_____

CHAPTER B

Work Log from Version History

These entries come from the repository history and can support the meeting record in the Project Progress Report Card (Annexure-3). Meeting dates and supervisor remarks are for the team and supervisor to fill in.

Date	Work completed
16 Sep 2026	v0.1: trajectory schema, agent protocol, harness, metrics, four failure modes, experiment runner, demo
19 Sep 2026	Synopsis first draft and first presentation draft
17 Oct 2026	v0.2: fault injection, repeated trials, horizon mode, scorers, suites, baseline agents, self-check, statistics, optimizer, store, reports, LLM adapter, CLI; 81 tests
17 Oct 2026	Design diagrams, results, research paper, progress presentation, this report